Weather Forecasting

Methods

Phase I

- **Generating Labels**
  - Sorted values in descending order → set top %5 to 1, bottom 95% to 0's

- **Generating Features**
  - Simultaneous indexing → realized each meteorological variable file has same dimensions (11,323 * 5,328), so we could index them for their days and place them in an array

Challenges

Phase I

- **Finding an efficient algorithm for computing samples M**
  - First attempt: the algorithm would run for 30 days! :(
  - Second attempt: the algorithm could run for 10 min! :)
- **Space → where to store all the info?**
  - Used .npy file format
  - Saved each feature in its own .npy file

Results

Phase I

- **Generated Features**
  - 11,295 features, each of size 5328*9*10
- **Generated Labels**
  - 11,295 label

Methods

Phase II

```
def SAOLA(data, C, trace=False):
    markov_blanket = []
    S_t_past = []

    for i,feature in enumerate(data):
        is_valid = True
        feaure_corr = Z(feature if trace else feature[2], C)
        if feaure_corr < 1.96:
            continue
        for j ,y in enumerate(S_t_past):
            y_corr = Z(y[2], C)
            f_y_corr = Z(feature if trace else feature[2], y[2])
            if y_corr > feaure_corr and f_y_corr > feaure_corr:
                is_valid = False
                break
            if feaure_corr > y_corr and f_y_corr > y_corr:
                S_t_past.pop(j)

        if is_valid:
            S_t_past.append((current_day, i+location_multiplier, feature)
                            if trace else feature)
            markov_blanket = S_t_past

    return markov_blanket
```

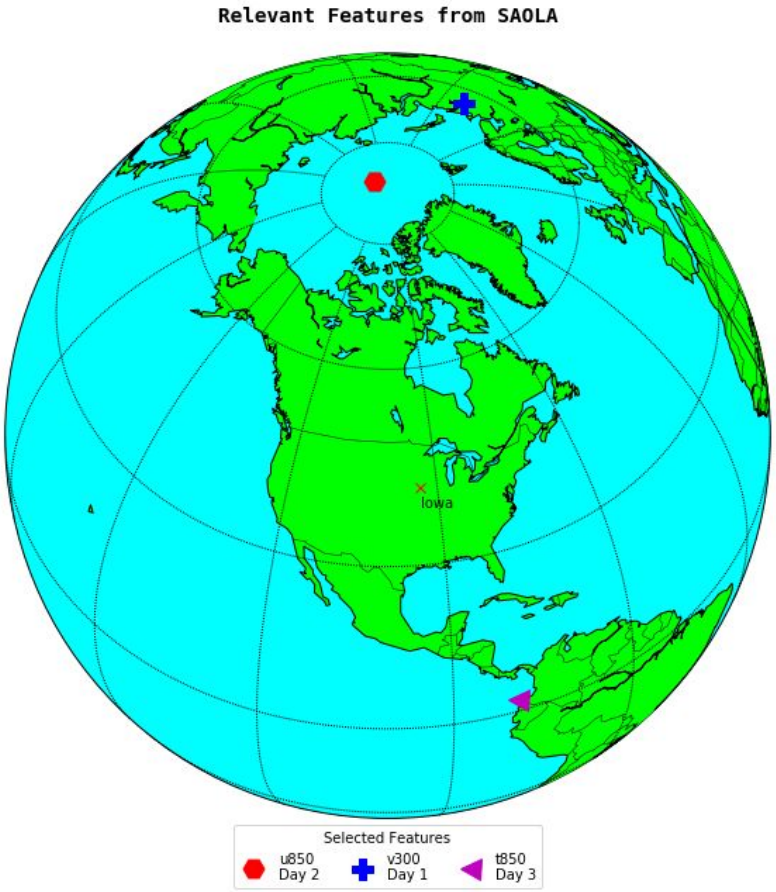
Challenges

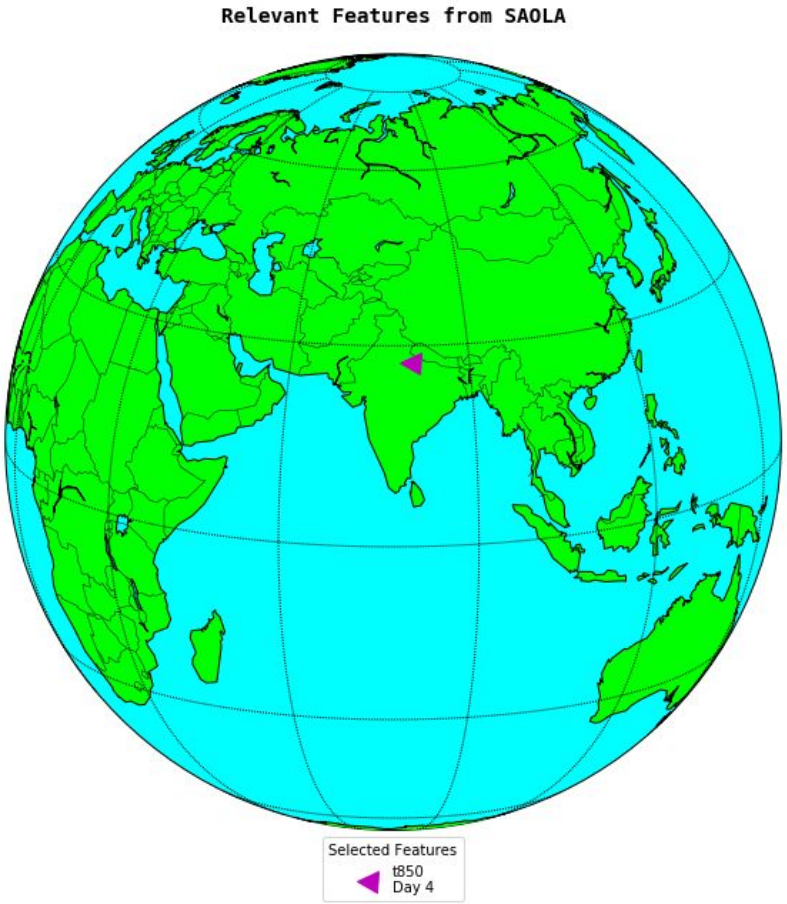
Phase II

- **Getting sufficient resultant features from SAOLA**
  - Implemented 2nd version of SAOLA and saved results in csv file
- **Plotting map of relevant features on map**
  - Problems when plotting on a flat map (Equidistant Cylindrical Projection)
  - Successfully plotted on orthographic projection

Results

Phase II





Results

Phase II

➔ 4 selected features (very close to what Yong's results were)

Day	Part	Latitude	Longitude
2	u850	87.50	130.00
1	v300	65.00	52.50
3	t850	2.50	280.00
4	t850	27.50	80.00

Challenges

Phase III

- ➔ Imbalanced data!
  - ◆ Got 98% accuracy for all classifiers, but found out that it was because the data was predicting all 0's
    - Implemented Upsampling
    - Implemented Downsampling

Results

Phase III

Algorithm	Cross-Validation (%)			Testing (%)		
	Imbalanced	Upsampled	Downsampled	Imbalanced	Upsampled	Downsampled
SVC	95.1	54.2	46.8	94.5	54.2	51.7
KNN	94.7	84.9	64.9	93.9	83.7	62.8
Naïve Bayes	94.1	59.7	60.1	93.6	59.2	59.3
Random Forest	94.9	93.8	70.1	94.1	93.1	69.5
Linear Discriminant Analysis	95.1	63.3	63.9	94.5	62.1	62.6
Multi-layer Perceptron	95.1	66.5	45.6	94.5	65.6	92.7

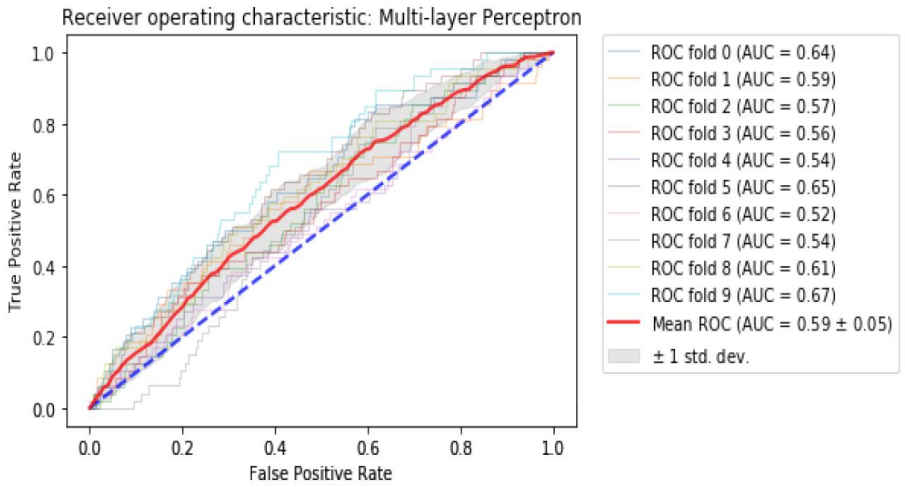
Green: AUC 80% (on Imbalance) Yellow : AUC > 80% Red: AUC > 78%

Bad: KNN, etc.

Good: Random Forest, Multi-layer Perceptron, etc.

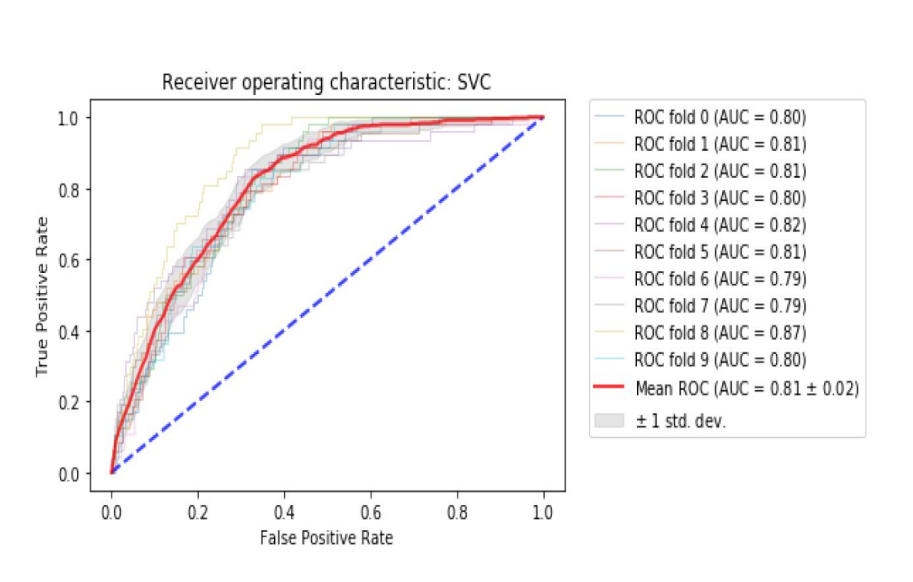
Phase III

ROC Graphs



Down-sampled (Bad)

Up-sampled (Good)



Questions!

Supplemental Material

(In case of Questions)

Main feature-building code:

```
for i in range(11295):
    print("Calculating feature sample ...", i)
    pw = pw_31[i:i+10]
    t850  =  t850_31[i:i+10]
    u300  =  u300_31[i:i+10]
    u850  =  u850_31[i:i+10]
    v300  =  v300_31[i:i+10]
    v850  =  v850_31[i:i+10]
    z1000 = z1000_31[i:i+10]
    z300  =  z300_31[i:i+10]
    z500  =  z500_31[i:i+10]
    M_1_10 = [] # clear one sample array
    mlist = [[],[],[],[],[],[],[],[],[],[]] #days 1 to 10
    updateM_1_10(mlist, M_1_10, pw, t850, u300, u850, v300, v850, z1000, z300, z500)
    M.append(np.concatenate(M_1_10, axis=0))
    M_1_10 = []
```

updateM_1_10 function:

```
def updateM_1_10(mlist, M_1_10, pw, t850, u300, u850, v300, v850, z1000, z300, z500):
    d = 0
    while d < 10:
        mlist[d].extend(np.concatenate([pw[d][4:],t850[d][4:],    u300[d][4:],
                                        u850[d][4:], v300[d][4:], v850[d][4:],
                                        z1000[d][4:], z300[d][4:], z500[d][4:]], axis=0))
        M_1_10.append(mlist[d])
        d += 1
```